Corrected Run Summary

Generated: 2026-05-15 10:59:34

Project folder: C:\Users\singb\Desktop\gpuu

This corrected PDF removes the accidental failed compile attempt that was present in the earlier report log. The report now shows only the clean compile and the successful simulation run.

Compile command: iverilog -g2012 -DTITAN_X5_USE_FPGA_DEMO_MODEL -o build\tb_top_arty_a7_corrected.vvp tb_top_arty_a7.v top_arty_a7.v

Run command: vvp build\tb_top_arty_a7_corrected.vvp

GTKWave input: tb_top_arty_a7_clean.vcd opened with tb_top_arty_a7.gtkw.

Result: PASS. The testbench completed with errors = 0, issues = 317, hits = 2, misses = 315, ready_seen = 1, and final_leds = 1111.

Scope of included source code: the Verilog testbench, FPGA wrapper/demo model, GTKWave save file, and Arty A7 constraints. The huge generated HDL file is listed in inventory but not dumped into the PDF because it is not part of this demo-model compile command and made the previous report unnecessarily noisy.

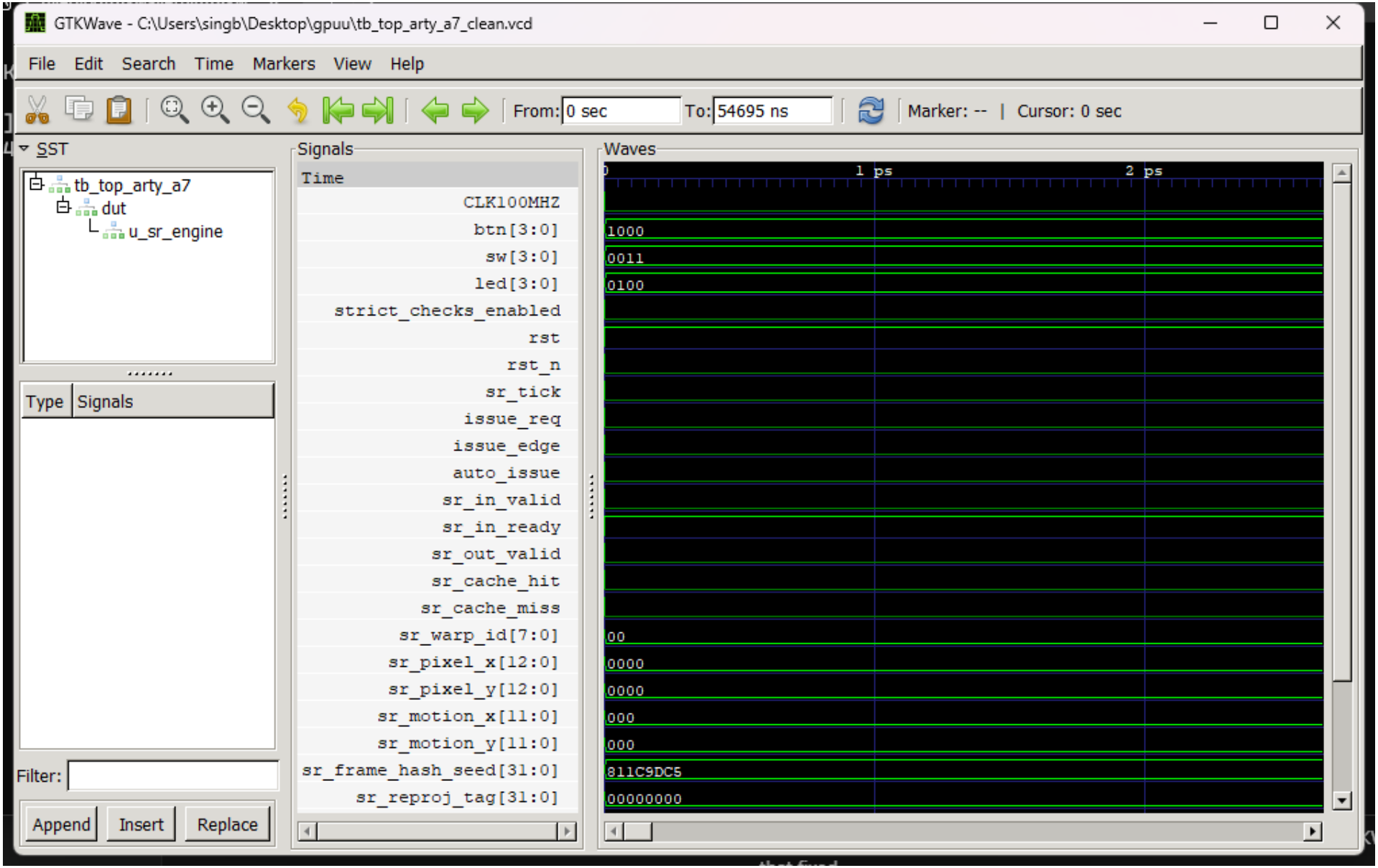
Clean Compile And Simulation Log

```
■Command: iverilog -g2012 -DTITAN_X5_USE_FPGA_DEMO_MODEL -o build\tb_top_arty_a7_corrected.vvp tb_top_arty_a7.v top_arty_a7.v
Exit code: 0
ICARUS COMPILE: PASS - 0 errors, 0 warnings


■Command: vvp build\tb_top_arty_a7_corrected.vvp
Exit code: 0
VCD info: dumpfile tb_top_arty_a7_clean.vcd opened for output.
[TB] Titan X5 Arty A7 wrapper Icarus test starting
[TB] cycles      = 5469
[TB] issues      = 317
[TB] hits        = 2
[TB] misses      = 315
[TB] ready_seen  = 1
[TB] errors      = 0
[TB] final_leds  = 1111
[TB] PASS: Arty wrapper generated warp issues and SR result traffic
C:\Users\singb\Desktop\gpuu\tb_top_arty_a7.v:207: $finish called at 54695000 (1ps)
```

GTKWave Screenshot

Waveform view captured from GTKWave using the generated VCD and project save file.



Top-Level Project Inventory

```
__pycache__                                folder -             modified 2026-04-29 18:33:08
arty_a7.xdc                                file   2,236 bytes    modified 2026-04-29 18:42:35
build                                      folder -             modified 2026-05-15 10:58:15
LINKEDIN_VIRAL_SEQUENCE.md                 file   5,001 bytes    modified 2026-04-29 18:42:35
README.md                                  file   8,358 bytes    modified 2026-04-29 18:42:35
SALES_OUTREACH_TEMPLATES.md                file   2,278 bytes    modified 2026-04-29 18:42:35
SELLING_PLAN.md                            file   3,096 bytes    modified 2026-04-29 18:31:59
simulate_titan_v19.py                      file   5,202 bytes    modified 2026-04-29 18:31:08
tb_top_arty_a7.gtkw                        file   1,426 bytes    modified 2026-05-08 15:48:46
tb_top_arty_a7.v                           file   6,539 bytes    modified 2026-05-08 15:47:07
tb_top_arty_a7.vcd                         file   923,938 bytes  modified 2026-05-08 15:03:31
tb_top_arty_a7_clean.vcd                   file   343,211 bytes  modified 2026-05-15 10:58:15
titan_v19_fixed.v                          file   557,970 bytes  modified 2026-04-29 18:33:10
Titan_X5_Apex_SR_Engine_Commercial_Package folder -             modified 2026-05-08 15:03:40
TITAN_X5_APEX_SR_ENGINE_DATASHEET.md       file   12,739 bytes   modified 2026-04-29 18:42:35
top_arty_a7.v                              file   12,335 bytes   modified 2026-05-08 15:08:35
Untitled-1.py                              file   7,951 bytes    modified 2026-03-28 15:48:00
Untitled-1_fixed.py                        file   9,553 bytes    modified 2026-04-29 18:33:02
vram_writes.csv                            file   9,876 bytes    modified 2026-04-29 18:33:09
vram_writes_full.csv                       file   565,468 bytes  modified 2026-04-29 18:33:31
wave.vcd                                   file   1,497,662 bytes modified 2026-03-28 16:30:26
wave_fixed.gtkw                            file   339 bytes      modified 2026-04-29 18:33:09
wave_fixed.vcd                             file   1,325,219 bytes modified 2026-04-29 18:33:09
```

Source: tb_top_arty_a7.v

```
0001: `timescale 1ns / 1ps
0002:
0003: module tb_top_arty_a7;
0004:     reg        CLK100MHZ = 1'b0;
0005:     reg  [3:0] sw        = 4'b0000;
0006:     reg  [3:0] btn       = 4'b0000;
0007:     wire [3:0] led;
0008:
0009:     integer cycle_count = 0;
0010:     integer issue_count = 0;
0011:     integer hit_count   = 0;
0012:     integer miss_count  = 0;
0013:     integer ready_seen  = 0;
0014:     integer error_count = 0;
0015:     reg     strict_checks_enabled = 1'b0;
0016:
0017:     top_arty_a7 #(
0018:         .CLK_DIV_BITS(4)
0019:     ) dut (
0020:         .CLK100MHZ(CLK100MHZ),
0021:         .sw(sw),
0022:         .btn(btn),
0023:         .led(led)
0024:     );
0025:
0026:     always #5 CLK100MHZ = ~CLK100MHZ;
0027:
0028:     task wait_cycles;
0029:         input integer cycles;
0030:         integer i;
0031:         begin
0032:             for (i = 0; i < cycles; i = i + 1) begin
0033:                 @(posedge CLK100MHZ);
0034:             end
0035:         end
0036:     endtask
0037:
0038:     task pulse_button0;
0039:         begin
0040:             btn[0] = 1'b1;
0041:             wait_cycles(40);
0042:             btn[0] = 1'b0;
0043:             wait_cycles(40);
0044:         end
0045:     endtask
0046:
0047:     always @(posedge CLK100MHZ) begin
0048:         cycle_count <= cycle_count + 1;
0049:
0050:         if (strict_checks_enabled) begin
0051:             if ((^led) === 1'bx) begin
0052:                 error_count <= error_count + 1;
0053:                 $fatal(1, "[TB] FAIL: led contains X/Z at %t", $time);
0054:             end
0055:
0056:             if ((^dut.rst) === 1'bx || (^dut.rst_n) === 1'bx) begin
0057:                 error_count <= error_count + 1;
0058:                 $fatal(1, "[TB] FAIL: reset path contains X/Z at %t", $time);
0059:             end
0060:
0061:             if ((^dut.sr_in_valid) === 1'bx || (^dut.sr_in_ready) === 1'bx ||
0062:                 (^dut.sr_out_valid) === 1'bx || (^dut.sr_cache_hit) === 1'bx ||
0063:                 (^dut.sr_cache_miss) === 1'bx) begin
```

Source: tb_top_arty_a7.v (continued)

```
0064:                 error_count <= error_count + 1;
0065:                 $fatal(1, "[TB] FAIL: ready/valid/cache status contains X/Z at %t", $time);
0066:             end
0067:
0068:             if (dut.sr_in_valid && dut.sr_in_ready) begin
0069:                 if ((^dut.sr_warp_id) === 1'bx ||
0070:                     (^dut.sr_pixel_x) === 1'bx ||
0071:                     (^dut.sr_pixel_y) === 1'bx ||
0072:                     (^dut.sr_motion_x) === 1'bx ||
0073:                     (^dut.sr_motion_y) === 1'bx ||
0074:                     (^dut.sr_frame_hash_seed) === 1'bx) begin
0075:                     error_count <= error_count + 1;
0076:                     $fatal(1, "[TB] FAIL: accepted input payload contains X/Z at %t", $time);
0077:                 end
0078:             end
0079:
0080:             if (dut.sr_out_valid) begin
0081:                 if ((^dut.sr_reproj_tag) === 1'bx ||
0082:                     (^dut.sr_confidence) === 1'bx) begin
0083:                     error_count <= error_count + 1;
0084:                     $fatal(1, "[TB] FAIL: output payload contains X/Z at %t", $time);
0085:                 end
0086:
0087:                 if (dut.sr_cache_hit && dut.sr_cache_miss) begin
0088:                     error_count <= error_count + 1;
0089:                     $fatal(1, "[TB] FAIL: cache_hit and cache_miss asserted together at %t", $time);
0090:                 end
0091:             end
0092:         end
0093:
0094:         if (dut.sr_in_valid && dut.sr_in_ready) begin
0095:             issue_count <= issue_count + 1;
0096:         end
0097:
0098:         if (dut.sr_out_valid && dut.sr_cache_hit) begin
0099:             hit_count <= hit_count + 1;
0100:         end
0101:
0102:         if (dut.sr_out_valid && dut.sr_cache_miss) begin
0103:             miss_count <= miss_count + 1;
0104:         end
0105:
0106:         if (dut.sr_in_ready) begin
0107:             ready_seen <= 1;
0108:         end
0109:     end
0110:
0111:     initial begin
0112:         sw  = 4'b0011;
0113:         btn = 4'b1000;
0114:
0115:         $timeformat(-9, 3, " ns", 10);
0116:         $dumpfile("tb_top_arty_a7_clean.vcd");
0117:         $dumpvars(0, CLK100MHZ);
0118:         $dumpvars(0, sw);
0119:         $dumpvars(0, btn);
0120:         $dumpvars(0, led);
0121:         $dumpvars(0, cycle_count);
0122:         $dumpvars(0, issue_count);
0123:         $dumpvars(0, hit_count);
0124:         $dumpvars(0, miss_count);
0125:         $dumpvars(0, ready_seen);
0126:         $dumpvars(0, error_count);
```

Source: tb_top_arty_a7.v (continued)

```
0127:         $dumpvars(0, strict_checks_enabled);
0128:         $dumpvars(0, dut.rst);
0129:         $dumpvars(0, dut.rst_n);
0130:         $dumpvars(0, dut.sr_tick);
0131:         $dumpvars(0, dut.issue_req);
0132:         $dumpvars(0, dut.issue_edge);
0133:         $dumpvars(0, dut.auto_issue);
0134:         $dumpvars(0, dut.sr_in_valid);
0135:         $dumpvars(0, dut.sr_in_ready);
0136:         $dumpvars(0, dut.sr_out_valid);
0137:         $dumpvars(0, dut.sr_cache_hit);
0138:         $dumpvars(0, dut.sr_cache_miss);
0139:         $dumpvars(0, dut.sr_warp_id);
0140:         $dumpvars(0, dut.sr_pixel_x);
0141:         $dumpvars(0, dut.sr_pixel_y);
0142:         $dumpvars(0, dut.sr_motion_x);
0143:         $dumpvars(0, dut.sr_motion_y);
0144:         $dumpvars(0, dut.sr_frame_hash_seed);
0145:         $dumpvars(0, dut.sr_reproj_tag);
0146:         $dumpvars(0, dut.sr_confidence);
0147:         $dumpvars(0, dut.u_sr_engine.last_tag);
0148:         $dumpvars(0, dut.u_sr_engine.next_tag);
0149:         $dumpvars(0, dut.u_sr_engine.reproj_tag);
0150:         $dumpvars(0, dut.u_sr_engine.confidence);
0151:
0152:         $display("[TB] Titan X5 Arty A7 wrapper Icarus test starting");
0153:         wait_cycles(20);
0154:
0155:         btn[3] = 1'b0;
0156:         wait_cycles(30);
0157:         strict_checks_enabled = 1'b1;
0158:
0159:         pulse_button0();
0160:         pulse_button0();
0161:
0162:         sw = 4'b1010;
0163:         pulse_button0();
0164:
0165:         btn[1] = 1'b1;
0166:         pulse_button0();
0167:         btn[1] = 1'b0;
0168:
0169:         btn[2] = 1'b1;
0170:         wait_cycles(5000);
0171:         btn[2] = 1'b0;
0172:         wait_cycles(100);
0173:
0174:         $display("[TB] cycles      = %0d", cycle_count);
0175:         $display("[TB] issues      = %0d", issue_count);
0176:         $display("[TB] hits        = %0d", hit_count);
0177:         $display("[TB] misses      = %0d", miss_count);
0178:         $display("[TB] ready_seen  = %0d", ready_seen);
0179:         $display("[TB] errors      = %0d", error_count);
0180:         $display("[TB] final_leds  = %b", led);
0181:
0182:         if (error_count != 0) begin
0183:             $fatal(1, "[TB] FAIL: expected zero errors");
0184:         end
0185:
0186:         if (!ready_seen) begin
0187:             $fatal(1, "[TB] FAIL: in_ready was never observed");
0188:         end
0189:
```

Source: tb_top_arty_a7.v (continued)

```
0190:         if (issue_count < 4) begin
0191:             $fatal(1, "[TB] FAIL: expected at least 4 accepted warp issues");
0192:         end
0193:
0194:         if ((hit_count + miss_count) < 4) begin
0195:             $fatal(1, "[TB] FAIL: expected at least 4 SR result events");
0196:         end
0197:
0198:         if (miss_count == 0) begin
0199:             $fatal(1, "[TB] FAIL: expected at least one cache miss");
0200:         end
0201:
0202:         if (hit_count == 0) begin
0203:             $display("[TB] NOTE: no cache hits observed in this deterministic traffic pattern");
0204:         end
0205:
0206:         $display("[TB] PASS: Arty wrapper generated warp issues and SR result traffic");
0207:         $finish;
0208:     end
0209: endmodule
```

Source: top_arty_a7.v

```
0001: // SPDX-License-Identifier: Apache-2.0
0002: // Copyright 2026 Titan X5 Apex SR Engine contributors.
0003: //
0004: // Top-level FPGA validation wrapper for the Titan X5 Apex SR Engine.
0005: // Target board: Digilent Arty A7-100T, 100 MHz onboard oscillator.
0006: //
0007: // Expected production SR Engine interface:
0008: //
0009: //   module titan_x5_apex_sr_engine #(
0010: //       parameter WARP_ID_W = 8,
0011: //       parameter COORD_W   = 13,
0012: //       parameter MV_W      = 12,
0013: //       parameter HASH_W    = 32,
0014: //       parameter META_W    = 16
0015: //   ) (
0016: //       input  wire                    clk,
0017: //       input  wire                    rst_n,
0018: //       input  wire                    in_valid,
0019: //       output wire                    in_ready,
0020: //       input  wire [WARP_ID_W-1:0]    in_warp_id,
0021: //       input  wire [COORD_W-1:0]      in_pixel_x,
0022: //       input  wire [COORD_W-1:0]      in_pixel_y,
0023: //       input  wire signed [MV_W-1:0]  in_motion_x,
0024: //       input  wire signed [MV_W-1:0]  in_motion_y,
0025: //       input  wire [HASH_W-1:0]       in_frame_hash_seed,
0026: //       output wire                    out_valid,
0027: //       input  wire                    out_ready,
0028: //       output wire                    cache_hit,
0029: //       output wire                    cache_miss,
0030: //       output wire [HASH_W-1:0]       reproj_tag,
0031: //       output wire [META_W-1:0]       confidence
0032: //   );
0033: //
0034: // If the production SR Engine is not yet present, compile this wrapper with
0035: // +define+TITAN_X5_USE_FPGA_DEMO_MODEL to use the synthesizable demo model at
0036: // the bottom of this file. Do not ship the demo model as licensed IP.
0037:
0038: `timescale 1ns / 1ps
0039:
0040: module top_arty_a7 #(
0041:     parameter integer CLK_DIV_BITS = 16
0042: ) (
0043:     input  wire        CLK100MHZ,
0044:     input  wire [3:0]  sw,
0045:     input  wire [3:0]  btn,
0046:     output wire [3:0]  led
0047: );
0048:
0049:     localparam integer WARP_ID_W = 8;
0050:     localparam integer COORD_W   = 13;
0051:     localparam integer MV_W      = 12;
0052:     localparam integer HASH_W    = 32;
0053:     localparam integer META_W    = 16;
0054:
0055:     wire clk = CLK100MHZ;
0056:
0057:     // Arty buttons are active high. btn[3] is the local validation reset.
0058:     reg [2:0] rst_sync;
0059:     initial rst_sync = 3'b111;
0060:
0061:     always @(posedge clk) begin
0062:         rst_sync <= {rst_sync[1:0], btn[3]};
0063:     end
```

Source: top_arty_a7.v (continued)

```
0064:
0065:     wire rst   = rst_sync[2];
0066:     wire rst_n = ~rst;
0067:
0068:     // Clock divider used as a validation-rate tick. The core remains on the
0069:     // board's global 100 MHz clock; ready/valid traffic is paced at a visible,
0070:     // board-safe rate so hit/miss LEDs can be inspected by eye.
0071:     reg [CLK_DIV_BITS-1:0] div_ctr;
0072:     initial div_ctr = {CLK_DIV_BITS{1'b0}};
0073:
0074:     always @(posedge clk) begin
0075:         if (rst) begin
0076:             div_ctr <= {CLK_DIV_BITS{1'b0}};
0077:         end else begin
0078:             div_ctr <= div_ctr + {{(CLK_DIV_BITS-1){1'b0}}, 1'b1};
0079:         end
0080:     end
0081:
0082:     wire sr_tick = &div_ctr;
0083:
0084:     // Synchronize and sample physical controls.
0085:     reg [3:0] sw_meta;
0086:     reg [3:0] sw_sync;
0087:     reg [3:0] btn_meta;
0088:     reg [3:0] btn_sync;
0089:
0090:     initial begin
0091:         sw_meta  = 4'b0000;
0092:         sw_sync  = 4'b0000;
0093:         btn_meta = 4'b0000;
0094:         btn_sync = 4'b0000;
0095:     end
0096:
0097:     always @(posedge clk) begin
0098:         sw_meta  <= sw;
0099:         sw_sync  <= sw_meta;
0100:         btn_meta <= btn;
0101:         btn_sync <= btn_meta;
0102:     end
0103:
0104:     // btn[0] issues a single warp request.
0105:     // btn[1] flips a hash salt bit to force cache-disagreement experiments.
0106:     // btn[2] enables continuous low-rate issue for hit-rate observation.
0107:     // btn[3] resets the wrapper and SR engine.
0108:     reg btn0_sample_d;
0109:     reg btn0_sample_q;
0110:     reg [7:0] issue_seq;
0111:     reg [7:0] frame_epoch;
0112:
0113:     initial begin
0114:         btn0_sample_d = 1'b0;
0115:         btn0_sample_q = 1'b0;
0116:         issue_seq     = 8'h00;
0117:         frame_epoch   = 8'h00;
0118:     end
0119:
0120:     always @(posedge clk) begin
0121:         if (rst) begin
0122:             btn0_sample_d <= 1'b0;
0123:             btn0_sample_q <= 1'b0;
0124:             issue_seq     <= 8'h00;
0125:             frame_epoch   <= 8'h00;
0126:         end else if (sr_tick) begin
```

Source: top_arty_a7.v (continued)

```
0127:             btn0_sample_d <= btn_sync[0];
0128:             btn0_sample_q <= btn0_sample_d;
0129:             if ((btn0_sample_d & ~btn0_sample_q) || btn_sync[2]) begin
0130:                 issue_seq   <= issue_seq + 8'h01;
0131:                 frame_epoch <= frame_epoch + {7'b0, btn_sync[1]};
0132:             end
0133:         end
0134:     end
0135:
0136:     wire issue_edge = sr_tick & (btn0_sample_d & ~btn0_sample_q);
0137:     wire auto_issue = sr_tick & btn_sync[2];
0138:     wire issue_req  = issue_edge | auto_issue;
0139:
0140:     wire                    sr_in_ready;
0141:     wire                    sr_out_valid;
0142:     wire                    sr_cache_hit;
0143:     wire                    sr_cache_miss;
0144:     wire [HASH_W-1:0]       sr_reproj_tag;
0145:     wire [META_W-1:0]       sr_confidence;
0146:
0147:     reg                     sr_in_valid;
0148:     reg [WARP_ID_W-1:0]     sr_warp_id;
0149:     reg [COORD_W-1:0]       sr_pixel_x;
0150:     reg [COORD_W-1:0]       sr_pixel_y;
0151:     reg signed [MV_W-1:0]   sr_motion_x;
0152:     reg signed [MV_W-1:0]   sr_motion_y;
0153:     reg [HASH_W-1:0]        sr_frame_hash_seed;
0154:
0155:     initial begin
0156:         sr_in_valid        = 1'b0;
0157:         sr_warp_id         = {WARP_ID_W{1'b0}};
0158:         sr_pixel_x         = {COORD_W{1'b0}};
0159:         sr_pixel_y         = {COORD_W{1'b0}};
0160:         sr_motion_x        = {MV_W{1'b0}};
0161:         sr_motion_y        = {MV_W{1'b0}};
0162:         sr_frame_hash_seed = 32'h811C9DC5;
0163:     end
0164:
0165:     wire [3:0] mode = sw_sync;
0166:
0167:     always @(posedge clk) begin
0168:         if (rst) begin
0169:             sr_in_valid        <= 1'b0;
0170:             sr_warp_id         <= {WARP_ID_W{1'b0}};
0171:             sr_pixel_x         <= {COORD_W{1'b0}};
0172:             sr_pixel_y         <= {COORD_W{1'b0}};
0173:             sr_motion_x        <= {MV_W{1'b0}};
0174:             sr_motion_y        <= {MV_W{1'b0}};
0175:             sr_frame_hash_seed <= 32'h811C9DC5;
0176:         end else begin
0177:             sr_in_valid <= 1'b0;
0178:
0179:             if (issue_req && sr_in_ready) begin
0180:                 sr_in_valid <= 1'b1;
0181:
0182:                 // Switches and buttons synthesize a repeatable "warp issue":
0183:                 // sw[3:0] select traffic mode, btn[1] perturbs the frame hash,
0184:                 // and issue_seq/frame_epoch provide motion over time.
0185:                 sr_warp_id  <= {frame_epoch[3:0], mode};
0186:                 sr_pixel_x  <= {1'b0, issue_seq, mode};
0187:                 sr_pixel_y  <= {1'b0, frame_epoch, issue_seq[3:0]};
0188:                 sr_motion_x <= mode[0] ? -12'sd2 : 12'sd2;
0189:                 sr_motion_y <= mode[1] ? -12'sd1 : 12'sd1;
```

Source: top_arty_a7.v (continued)

```
0190:                 sr_frame_hash_seed <= 32'h811C9DC5
0191:                                       ^ {24'h000000, issue_seq}
0192:                                       ^ {16'h0000, frame_epoch, mode}
0193:                                       ^ (btn_sync[1] ? 32'h01000193 : 32'h00000000);
0194:             end
0195:         end
0196:     end
0197:
0198: `ifdef TITAN_X5_USE_FPGA_DEMO_MODEL
0199:     titan_x5_apex_sr_engine_demo_model #(
0200:         .WARP_ID_W(WARP_ID_W),
0201:         .COORD_W(COORD_W),
0202:         .MV_W(MV_W),
0203:         .HASH_W(HASH_W),
0204:         .META_W(META_W)
0205:     ) u_sr_engine (
0206:         .clk(clk),
0207:         .rst_n(rst_n),
0208:         .in_valid(sr_in_valid),
0209:         .in_ready(sr_in_ready),
0210:         .in_warp_id(sr_warp_id),
0211:         .in_pixel_x(sr_pixel_x),
0212:         .in_pixel_y(sr_pixel_y),
0213:         .in_motion_x(sr_motion_x),
0214:         .in_motion_y(sr_motion_y),
0215:         .in_frame_hash_seed(sr_frame_hash_seed),
0216:         .out_valid(sr_out_valid),
0217:         .out_ready(1'b1),
0218:         .cache_hit(sr_cache_hit),
0219:         .cache_miss(sr_cache_miss),
0220:         .reproj_tag(sr_reproj_tag),
0221:         .confidence(sr_confidence)
0222:     );
0223: `else
0224:     titan_x5_apex_sr_engine #(
0225:         .WARP_ID_W(WARP_ID_W),
0226:         .COORD_W(COORD_W),
0227:         .MV_W(MV_W),
0228:         .HASH_W(HASH_W),
0229:         .META_W(META_W)
0230:     ) u_sr_engine (
0231:         .clk(clk),
0232:         .rst_n(rst_n),
0233:         .in_valid(sr_in_valid),
0234:         .in_ready(sr_in_ready),
0235:         .in_warp_id(sr_warp_id),
0236:         .in_pixel_x(sr_pixel_x),
0237:         .in_pixel_y(sr_pixel_y),
0238:         .in_motion_x(sr_motion_x),
0239:         .in_motion_y(sr_motion_y),
0240:         .in_frame_hash_seed(sr_frame_hash_seed),
0241:         .out_valid(sr_out_valid),
0242:         .out_ready(1'b1),
0243:         .cache_hit(sr_cache_hit),
0244:         .cache_miss(sr_cache_miss),
0245:         .reproj_tag(sr_reproj_tag),
0246:         .confidence(sr_confidence)
0247:     );
0248: `endif
0249:
0250:     // Stretch hit/miss pulses so they are visible on the board LEDs.
0251:     reg [19:0] hit_hold;
0252:     reg [19:0] miss_hold;
```

Source: top_arty_a7.v (continued)

```
0253:     reg [19:0] issue_hold;
0254:
0255:     initial begin
0256:         hit_hold   = 20'd0;
0257:         miss_hold  = 20'd0;
0258:         issue_hold = 20'd0;
0259:     end
0260:
0261:     always @(posedge clk) begin
0262:         if (rst) begin
0263:             hit_hold   <= 20'd0;
0264:             miss_hold  <= 20'd0;
0265:             issue_hold <= 20'd0;
0266:         end else begin
0267:             if (sr_out_valid && sr_cache_hit) begin
0268:                 hit_hold <= 20'hFFFFF;
0269:             end else if (hit_hold != 20'd0) begin
0270:                 hit_hold <= hit_hold - 20'd1;
0271:             end
0272:
0273:             if (sr_out_valid && sr_cache_miss) begin
0274:                 miss_hold <= 20'hFFFFF;
0275:             end else if (miss_hold != 20'd0) begin
0276:                 miss_hold <= miss_hold - 20'd1;
0277:             end
0278:
0279:             if (sr_in_valid && sr_in_ready) begin
0280:                 issue_hold <= 20'h7FFFF;
0281:             end else if (issue_hold != 20'd0) begin
0282:                 issue_hold <= issue_hold - 20'd1;
0283:             end
0284:         end
0285:     end
0286:
0287:     assign led[0] = (hit_hold  != 20'd0);            // Cache hit
0288:     assign led[1] = (miss_hold != 20'd0);            // Cache miss
0289:     assign led[2] = sr_in_ready;                     // Core accepts requests
0290:     assign led[3] = (issue_hold != 20'd0) ^ btn_sync[2]; // Warp issue activity
0291:
0292: endmodule
0293:
0294: `ifdef TITAN_X5_USE_FPGA_DEMO_MODEL
0295: module titan_x5_apex_sr_engine_demo_model #(
0296:     parameter integer WARP_ID_W = 8,
0297:     parameter integer COORD_W   = 13,
0298:     parameter integer MV_W      = 12,
0299:     parameter integer HASH_W    = 32,
0300:     parameter integer META_W    = 16
0301: ) (
0302:     input  wire                    clk,
0303:     input  wire                    rst_n,
0304:     input  wire                    in_valid,
0305:     output wire                    in_ready,
0306:     input  wire [WARP_ID_W-1:0]    in_warp_id,
0307:     input  wire [COORD_W-1:0]      in_pixel_x,
0308:     input  wire [COORD_W-1:0]      in_pixel_y,
0309:     input  wire signed [MV_W-1:0]  in_motion_x,
0310:     input  wire signed [MV_W-1:0]  in_motion_y,
0311:     input  wire [HASH_W-1:0]       in_frame_hash_seed,
0312:     output reg                     out_valid,
0313:     input  wire                    out_ready,
0314:     output reg                     cache_hit,
0315:     output reg                     cache_miss,
```

Source: top_arty_a7.v (continued)

```
0316:     output reg [HASH_W-1:0]        reproj_tag,
0317:     output reg [META_W-1:0]        confidence
0318: );
0319:
0320:     assign in_ready = out_ready;
0321:
0322:     function [31:0] fnv1a32;
0323:         input [31:0] seed;
0324:         input [31:0] data;
0325:         reg [31:0] h;
0326:         begin
0327:             h = seed ^ data[7:0];
0328:             h = h * 32'h01000193;
0329:             h = h ^ data[15:8];
0330:             h = h * 32'h01000193;
0331:             h = h ^ data[23:16];
0332:             h = h * 32'h01000193;
0333:             h = h ^ data[31:24];
0334:             h = h * 32'h01000193;
0335:             fnv1a32 = h;
0336:         end
0337:     endfunction
0338:
0339:     reg [31:0] last_tag;
0340:     reg [31:0] next_tag;
0341:
0342:     initial begin
0343:         out_valid  = 1'b0;
0344:         cache_hit  = 1'b0;
0345:         cache_miss = 1'b0;
0346:         reproj_tag = {HASH_W{1'b0}};
0347:         confidence = {META_W{1'b0}};
0348:         last_tag   = 32'h00000000;
0349:         next_tag   = 32'h00000000;
0350:     end
0351:
0352:     always @* begin
0353:         next_tag = fnv1a32(
0354:             in_frame_hash_seed,
0355:             {in_warp_id, in_pixel_x[7:0], in_pixel_y[7:0], in_motion_x[3:0], in_motion_y[3:0]}
0356:         );
0357:     end
0358:
0359:     always @(posedge clk) begin
0360:         if (!rst_n) begin
0361:             out_valid  <= 1'b0;
0362:             cache_hit  <= 1'b0;
0363:             cache_miss <= 1'b0;
0364:             reproj_tag <= {HASH_W{1'b0}};
0365:             confidence <= {META_W{1'b0}};
0366:             last_tag   <= 32'h00000000;
0367:         end else begin
0368:             out_valid <= 1'b0;
0369:             if (in_valid && in_ready) begin
0370:                 out_valid  <= 1'b1;
0371:                 reproj_tag <= next_tag;
0372:                 cache_hit  <= (next_tag[7:4] == last_tag[7:4]);
0373:                 cache_miss <= (next_tag[7:4] != last_tag[7:4]);
0374:                 confidence <= (next_tag[7:4] == last_tag[7:4]) ? 16'hE400 : 16'h4100;
0375:                 last_tag   <= next_tag;
0376:             end
0377:         end
0378:     end
```

Source: top_arty_a7.v (continued)

```
0379:
0380: endmodule
0381: `endif
```

Source: tb_top_arty_a7.gtkw

```
0001: [dumpfile] "C:\Users\singb\Desktop\gpuu\tb_top_arty_a7_clean.vcd"
0002: [dumpfile_size] 343211
0003: [savefile] "C:\Users\singb\Desktop\gpuu\tb_top_arty_a7.gtkw"
0004: [timestart] 0
0005: [treeopen] tb_top_arty_a7.
0006: [treeopen] tb_top_arty_a7.dut.
0007: [treeopen] tb_top_arty_a7.dut.u_sr_engine.
0008: @28
0009: tb_top_arty_a7.CLK100MHZ
0010: tb_top_arty_a7.btn[3:0]
0011: tb_top_arty_a7.sw[3:0]
0012: tb_top_arty_a7.led[3:0]
0013: tb_top_arty_a7.strict_checks_enabled
0014: @22
0015: tb_top_arty_a7.cycle_count[31:0]
0016: tb_top_arty_a7.issue_count[31:0]
0017: tb_top_arty_a7.hit_count[31:0]
0018: tb_top_arty_a7.miss_count[31:0]
0019: tb_top_arty_a7.ready_seen[31:0]
0020: tb_top_arty_a7.error_count[31:0]
0021: @28
0022: tb_top_arty_a7.dut.rst
0023: tb_top_arty_a7.dut.rst_n
0024: tb_top_arty_a7.dut.sr_tick
0025: tb_top_arty_a7.dut.issue_req
0026: tb_top_arty_a7.dut.issue_edge
0027: tb_top_arty_a7.dut.auto_issue
0028: tb_top_arty_a7.dut.sr_in_valid
0029: tb_top_arty_a7.dut.sr_in_ready
0030: tb_top_arty_a7.dut.sr_out_valid
0031: tb_top_arty_a7.dut.sr_cache_hit
0032: tb_top_arty_a7.dut.sr_cache_miss
0033: @22
0034: tb_top_arty_a7.dut.sr_warp_id[7:0]
0035: tb_top_arty_a7.dut.sr_pixel_x[12:0]
0036: tb_top_arty_a7.dut.sr_pixel_y[12:0]
0037: tb_top_arty_a7.dut.sr_motion_x[11:0]
0038: tb_top_arty_a7.dut.sr_motion_y[11:0]
0039: tb_top_arty_a7.dut.sr_frame_hash_seed[31:0]
0040: tb_top_arty_a7.dut.sr_reproj_tag[31:0]
0041: tb_top_arty_a7.dut.sr_confidence[15:0]
0042: @22
0043: tb_top_arty_a7.dut.u_sr_engine.last_tag[31:0]
0044: tb_top_arty_a7.dut.u_sr_engine.next_tag[31:0]
0045: tb_top_arty_a7.dut.u_sr_engine.reproj_tag[31:0]
0046: tb_top_arty_a7.dut.u_sr_engine.confidence[15:0]
```

Source: arty_a7.xdc

```
0001: ## SPDX-License-Identifier: Apache-2.0
0002: ## Titan X5 Apex SR Engine - Arty A7-100T FPGA validation constraints
0003: ## Board: Digilent Arty A7-100T Rev. D/E
0004: ## Top-level: top_arty_a7
0005: ##
0006: ## Pin assignments use the Digilent Arty-A7-100-Master.xdc mapping.
0007:
0008: ## 100 MHz onboard oscillator
0009: set_property -dict { PACKAGE_PIN E3 IOSTANDARD LVCMOS33 } [get_ports { CLK100MHZ }]
0010: create_clock -add -name sys_clk_pin -period 10.000 -waveform {0.000 5.000} [get_ports { CLK100MHZ }]
0011:
0012: ## Slide switches: traffic mode, hash/motion mode bits
0013: set_property -dict { PACKAGE_PIN A8  IOSTANDARD LVCMOS33 } [get_ports { sw[0] }]
0014: set_property -dict { PACKAGE_PIN C11 IOSTANDARD LVCMOS33 } [get_ports { sw[1] }]
0015: set_property -dict { PACKAGE_PIN C10 IOSTANDARD LVCMOS33 } [get_ports { sw[2] }]
0016: set_property -dict { PACKAGE_PIN A10 IOSTANDARD LVCMOS33 } [get_ports { sw[3] }]
0017:
0018: ## Push buttons:
0019: ## btn[0] = single warp issue
0020: ## btn[1] = hash salt perturbation
0021: ## btn[2] = continuous issue mode
0022: ## btn[3] = validation reset
0023: set_property -dict { PACKAGE_PIN D9 IOSTANDARD LVCMOS33 } [get_ports { btn[0] }]
0024: set_property -dict { PACKAGE_PIN C9 IOSTANDARD LVCMOS33 } [get_ports { btn[1] }]
0025: set_property -dict { PACKAGE_PIN B9 IOSTANDARD LVCMOS33 } [get_ports { btn[2] }]
0026: set_property -dict { PACKAGE_PIN B8 IOSTANDARD LVCMOS33 } [get_ports { btn[3] }]
0027:
0028: ## User LEDs:
0029: ## led[0] = SR cache hit pulse
0030: ## led[1] = SR cache miss pulse
0031: ## led[2] = core input ready
0032: ## led[3] = warp issue activity
0033: set_property -dict { PACKAGE_PIN H5  IOSTANDARD LVCMOS33 } [get_ports { led[0] }]
0034: set_property -dict { PACKAGE_PIN J5  IOSTANDARD LVCMOS33 } [get_ports { led[1] }]
0035: set_property -dict { PACKAGE_PIN T9  IOSTANDARD LVCMOS33 } [get_ports { led[2] }]
0036: set_property -dict { PACKAGE_PIN T10 IOSTANDARD LVCMOS33 } [get_ports { led[3] }]
0037:
0038: ## Conservative electrical defaults for board-level validation.
0039: set_property DRIVE 8 [get_ports { led[*] }]
0040: set_property SLEW SLOW [get_ports { led[*] }]
0041:
0042: ## Input controls are asynchronous to CLK100MHZ and are synchronized in RTL.
0043: set_false_path -from [get_ports { sw[*] }]
0044: set_false_path -from [get_ports { btn[*] }]
0045:
0046: ## Vivado bitstream generation convenience.
0047: set_property CONFIG_VOLTAGE 3.3 [current_design]
0048: set_property CFGBVS VCCO [current_design]
```